

Typing the Bomb

Technical Documentation & Architecture Report

Engine Version: 1.0.0 • **Language Standard:** ISO/IEC C99 • **Platform Support:** macOS, Linux, Windows

1. Executive Summary & Overview

"Typing the Bomb" is a high-performance, real-time arcade simulation engine executed entirely within a standard terminal environment. Modeled after classic tactical typing games, the system spawns an aerial asset (a plane) moving horizontally across the upper boundary of the display matrix, systematically deploying ordnance (bombs) dropping vertically toward a populated target sector (a city).

Each projectile is programmatically bound to a dynamically allocated codeword extracted from a filtered dictionary. The player acts as the defusal system, required to process and type the string sequence letter-by-letter before the entity's coordinate reaches the threshold of the base layer. The core software architecture emphasizes memory efficiency, rigid state tracking, cross-platform hardware abstraction, and deterministic rendering loops.

2. System Architecture & State Tracking

2.1 Monolithic Architecture Allocation

The runtime core is orchestrated within a consolidated source structure (main.c). This tactical choice eliminates inter-module lookup overhead, optimizes inline compile optimizations, and provides a streamlined build profile across diverse toolchains. The layout segregates clearly into preprocessor system wrappers, structured state definitions, static I/O pipeline buffers, and the primary deterministic loop machine.

2.2 Pointer-Free Game State Serialization

To facilitate an instantaneous, zero-overhead storage model, the entire volatile state of the application is encapsulated inside a strict, pointer-free data structure: GameState.

Architectural Paradigm: By exclusively using primitive numeric types and fixed-size contiguous matrices, the runtime state guarantees binary predictability. It completely avoids memory fragmentation and dynamic address allocation hazards.

This specific constraint enables native disk serialization. Saving or restoring a session bypasses complex deep-copy or structural parsing routines; instead, the memory region is directly written to or read from disk using low-level ANSI C file streams:

```
/* Low-overhead atomic snapshot storage example */
fwrite(&currentGame, sizeof(GameState), 1, filePointer);
fread(&currentGame, sizeof(GameState), 1, filePointer);
```

3. Core Engineering Subsystems

3.1 Asynchronous Non-Blocking Input & Clock Regulation

Real-time terminal responsiveness requires capturing user inputs without blocking the primary execution branch. To solve this across heterogeneous environments, a low-level abstraction layer wrapping OS-specific calls was developed:

- **Windows Environments:** Leverages the native conio.h runtime map, routing queries directly through efficient kernel wrappers (`_kbhit()` and `_getch()`).
- **POSIX/UNIX Environments (macOS & Linux):** Manually manipulates the terminal driver state via the `termios.h` framework. The system disables canonical mode (unbuffered input line processing) and echo modes, switching the standard input file descriptor (`STDIN_FILENO`) to an instantaneous polling configuration.

Clock precision and uniform execution velocities are sustained using a deterministic frame delay scheme. The main loop forces an execution throttle of approximately 60 milliseconds per cycle, generating a stable target rate of ~16.6 updates per second.

3.2 Double-Buffered Console Graphics Engine

Directly drawing text artifacts to a standard terminal display stream causes intense visual flickering and artifact trailing due to varying I/O speeds. To resolve this, the system implements a software-level Double-Buffering Rendering Pipeline.

Rather than outputting strings piecemeal, the internal rendering logic maps all updates onto a discrete, two-dimensional character matrix acting as a VRAM frame buffer: `screen[SCREEN_H][SCREEN_W]`. Once the composite layout is built, a targeted terminal reset control is executed, and the entire buffer block is written to standard output in a single continuous stream. This dramatically cuts down on system draw calls and results in perfectly smooth animation updates.

3.3 Algorithmic Scoring & Progression Mechanics

The engine evaluates input accuracy using a raw performance metric computed instantly per character transaction:

$$\text{Score} = \text{Letters}_{\text{Correct}} - \text{Mistakes}_{\text{Penalized}}$$

Difficulty scaling operates dynamically via the projectile generator function. Progression tracks historical destruction values; for every 5 entities eliminated, the system increments the length constraint of newly instantiated codewords by 1 unit, starting from a baseline of 3 characters up to a ceiling of 10 characters.

4. Resource Pipeline & File I/O

The asset framework handles text assets and raw binary storage through three dedicated subsystems:

Resource Type	Target Asset	Processing Protocol
ASCII Sprites	plane.txt, bomb.txt, city.txt	Parses structural design blocks, automatically stripping cross-platform carriage returns () and trailing newlines ().
Codeword Pool	codewords.txt	Validates structural string arrays, filtering out non-alphabetic elements and truncating buffers to design specifications.
State Snapshots	User-specified binary blob	Direct block-level memory dump of the active struct state for instantaneous session restoration.

5. Build Automation & Cross-Platform Toolchain

Symmetric compilation and execution environments are established natively using a unified CMake automation script. To ensure that external graphic assets and dictionary maps remain persistently linkable regardless of where the output binary resides, the build automation script handles resource copying explicitly:

```
cmake_minimum_required(VERSION 3.10)
project(typing_bomb_game C)
set(CMAKE_C_STANDARD 99)
add_executable(main main.c)

# Automate resource asset deployment to target build directory
file(COPY plane.txt bomb.txt city.txt codewords.txt
      DESTINATION ${CMAKE_BINARY_DIR})
```